

Lab Activity 5

Objectives

- To design a dashboard and redesign the layout pages.
- To implement and utilize the MUI Component Library
- To revise the code relatively

Materials Needed:

- A computer with Node.js and React JS installed.
- IDE: VS Code, Github Codespaces, or Code Sandbox

Instructions:

Replicate the code snippet.

Follow and implement the enhancement instructions.

Web App Initialization:

Sample design pattern [component based]

```
robles-webprog/
├── README.md
├── project.drawio
├── robles-client/
│   ├── eslint.config.js
│   ├── index.html
│   ├── package-lock.json
│   ├── package.json
│   ├── vite.config.js
│   ├── public/
│   │   └── vite.svg
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       ├── assets/
│       │   ├── article-content.js
│       │   ├── hero.png
│       │   ├── react.svg
│       │   ├── vite.svg
│       │   └── styles/
│       │       └── index.css
│       ├── components/
│       │   ├── ArticleList.jsx
│       │   ├── Button.jsx
│       │   ├── Footer.jsx
│       │   └── NavBar.jsx
│       ├── layouts/
│       │   ├── AuthLayout.jsx
│       │   ├── DashLayout.jsx
│       │   └── Layout.jsx
│       └── pages/
│           ├── NotFoundPage.jsx
│           ├── AuthPages/
│           │   ├── SignInPage.jsx
│           │   └── SignUpPage.jsx
│           ├── DashboardPages/
│           │   ├── DashboardPage.jsx
│           │   ├── ReportsPage.jsx
│           │   └── UsersPage.jsx
│           └── LandingPages/
│               ├── AboutPage.jsx
│               ├── ArticleListPage.jsx
│               ├── ArticlePage.jsx
│               └── HomePage.jsx
```



Code Snippets

App.jsx [just add routes to locate the new pages]

```
const routes = [
  {
    path: '/',
    element: <Layout />,
    errorElement: <NotFoundPage />,
    children: [
      {
        path: '',
        element: <HomePage />,
      },
      {
        path: 'about',
        element: <AboutPage />,
      },
      {
        path: 'articles',
        element: <ArticleListPage />,
      },
      {
        path: 'articles/:name',
        element: <ArticlePage />,
      },
    ],
  },
  {
    path: "auth/",
    element: <AuthLayout />,
    errorElement: <NotFoundPage />,
    children: [
      {
        path: "signin",
        element: <SignInPage />,
      },
      {
        path: "signup",
        element: <SignUpPage />,
      },
    ],
  },
  {
    path: "dashboard/",
    element: <DashLayout />,
    errorElement: <NotFoundPage />,
    children: [
      {
        path: "",
        element: <DashboardPage />,
      },
      {
        path: "reports",
        element: <ReportsPage />,
      },
      {
        path: "users",
        element: <UsersPage />,
      },
    ],
  },
],
];
```



layouts/
DashLayout.jsx

```
import React, { useState } from "react";
import { Outlet, Link, useLocation, useNavigate } from "react-router-dom";
import { styled, useTheme, alpha } from "@mui/material/styles";
import Box from "@mui/material/Box";
import MuiDrawer from "@mui/material/Drawer";
import MuiAppBar from "@mui/material/AppBar";
import Toolbar from "@mui/material/Toolbar";
import List from "@mui/material/List";
import CssBaseline from "@mui/material/CssBaseline";
import Typography from "@mui/material/Typography";
import Divider from "@mui/material/Divider";
import IconButton from "@mui/material/IconButton";
import MenuIcon from "@mui/icons-material/Menu";
import SearchIcon from "@mui/icons-material/Search";
import InputBase from "@mui/material/InputBase";
import ChevronLeftIcon from "@mui/icons-material/ChevronLeft";
import ChevronRightIcon from "@mui/icons-material/ChevronRight";
import ListItem from "@mui/material/ListItem";
import ListItemButton from "@mui/material/ListItemButton";
import ListItemIcon from "@mui/material/ListItemIcon";
import ListItemText from "@mui/material/ListItemText";
import DashboardIcon from "@mui/icons-material/Dashboard";
import PeopleIcon from "@mui/icons-material/People";
import AssessmentIcon from "@mui/icons-material/Assessment";
import Button from "@mui/material/Button";
import MenuOpenIcon from "@mui/icons-material/MenuOpen";
import ArticleIcon from "@mui/icons-material/Article";

const drawerWidth = 240;
const dashboardNavItems = [
  {
    label: "Dashboard",
    title: "Dashboard",
    to: "/dashboard",
    icon: DashboardIcon,
  },
  {
    label: "Reports",
    title: "Reports",
    to: "/dashboard/reports",
    icon: AssessmentIcon,
  },
  {
    label: "Users",
    title: "Users",
    to: "/dashboard/users",
    icon: PeopleIcon,
  },
];

const openedMixin = (theme) => ({
  width: drawerWidth,
  transition: theme.transitions.create("width", {
    easing: theme.transitions.easing.sharp,
    duration: theme.transitions.duration.enteringScreen,
  }),
  overflowX: "hidden",
});

const closedMixin = (theme) => ({
  transition: theme.transitions.create("width", {
    easing: theme.transitions.easing.sharp,
    duration: theme.transitions.duration.leavingScreen,
  }),
  overflowX: "hidden",
  width: `calc(${theme.spacing(7)} + 1px)`,
  [theme.breakpoints.up("sm")]: {
    width: `calc(${theme.spacing(8)} + 1px)`,
  },
});

const DrawerHeader = styled("div")(({ theme }) => ({
  display: "flex",
  alignItems: "center",
  justifyContent: "flex-end",
  padding: theme.spacing(0, 1),
  ...theme.mixins.toolbar,
}));

const AppBar = styled(MuiAppBar, {
  shouldForwardProp: (prop) => prop !== "open",
})(({ theme, open }) => ({
  zIndex: theme.zIndex.drawer + 1,
  transition: theme.transitions.create(["width", "margin"], {
    easing: theme.transitions.easing.sharp,
    duration: theme.transitions.duration.leavingScreen,
  }),
  ...(open && {
    marginLeft: drawerWidth,
    width: `calc(100% - ${drawerWidth}px)`,
    transition: theme.transitions.create(["width", "margin"], {
      easing: theme.transitions.easing.sharp,
      duration: theme.transitions.duration.enteringScreen,
    }),
  }),
}));

const Drawer = styled(MuiDrawer, {
  shouldForwardProp: (prop) => prop !== "open",
})(({ theme, open }) => ({
  width: drawerWidth,
  flexShrink: 0,
  whiteSpace: "nowrap",
  boxSizing: "border-box",
  ...(open && {
    ...openedMixin(theme),
    "& .MuiDrawer-paper": openedMixin(theme),
  }),
  ...(!open && {
    ...closedMixin(theme),
    "& .MuiDrawer-paper": closedMixin(theme),
  }),
}));

const SearchIconWrapper = styled("div")(({ theme }) => ({
  padding: theme.spacing(0, 2),
  height: "100%",
  position: "absolute",
  pointerEvents: "none",
  display: "flex",
  alignItems: "center",
  justifyContent: "center",
}));

const Search = styled("div")(({ theme }) => ({
  position: "relative",
  borderRadius: theme.shape.borderRadius,
  backgroundColor: alpha(theme.palette.common.white, 0.15),
  "&:hover": {
    backgroundColor: alpha(theme.palette.common.white, 0.25),
  },
  marginRight: theme.spacing(2),
  marginLeft: 0,
  width: "100%",
  [theme.breakpoints.up("sm")]: {
    marginLeft: theme.spacing(3),
    width: "auto",
  },
}));

const StyledInputBase = styled(InputBase)(({ theme }) => ({
  color: "inherit",
  "& .MuiInputBase-input": {
    padding: theme.spacing(1, 1, 0),
    // vertical padding + font size from searchIcon
    paddingLeft: `calc(1em + ${theme.spacing(4)})`,
    transition: theme.transitions.create("width"),
    width: "100%",
    [theme.breakpoints.up("md")]: {
      width: "20ch",
    },
  },
}));

const getPageTitle = (pathname) =>
  dashboardNavItems.find(({ to }) => to === pathname)?.title ?? "Welcome";
```



```
const DashLayout = () => {
  const theme = useTheme();
  const [open, setOpen] = useState(false);
  const location = useLocation();
  const pageTitle = getPageTitle(location.pathname);
  const navigate = useNavigate();

  const handleDrawerOpen = () => {
    setOpen(true);
  };

  const handleDrawerClose = () => {
    setOpen(false);
  };

  const handleLogout = () => {
    navigate("/");
  };

  return (
    <Box sx={{ display: "flex" }}>
      <CssBaseline />
      <AppBar position="fixed" open={open}>
        <Toolbar>
          <IconButton
            color="inherit"
            aria-label="open drawer"
            // onClick={open ? handleDrawerClose : handleDrawerOpen}
            edge="start"
            // sx={{ marginRight: 5, ...(open && { display: 'none' }) }}
            sx={{ marginRight: 5, ...open }}
          >
            {open ? <MenuOpenIcon /> : <MenuIcon />}
          </IconButton>
          <Typography
            variant="h6"
            noWrap
            component="div"
            sx={{ flexGrow: 1 }}
          >
            {pageTitle}
          </Typography>
          <Search />
          <SearchIconWrapper>
            <SearchIcon />
          </SearchIconWrapper>
          <StyledInputBase
            placeholder="Search..."
            inputProps={{ "aria-label": "search" }}
          />
          <Button color="inherit" variant="outlined" onClick={handleLogout}>
            Logout
          </Button>
        </Toolbar>
      </AppBar>
      <Drawer variant="permanent" open={open}>
        <DrawerHeader>
          <IconButton onClick={handleDrawerClose}>
            {theme.direction === "rtl" ? (
              <ChevronRightIcon />
            ) : (
              <ChevronLeftIcon />
            )}
          </IconButton>
        </DrawerHeader>
        <Divider />
        <List>
          {dashboardNavItems.map(({ label, to, icon: Icon }) => (
            <ListItem key={to} disablePadding sx={{ display: "block" }}>
              <ListItemButton
                component={Link}
                to={to}
                selected={location.pathname === to}
                sx={{
                  minHeight: 48,
                  px: 2.5,
                  justifyContent: open ? "initial" : "center",
                }}
              >
                <ListItemIcon>
                  <Icon />
                </ListItemIcon>
                <ListItemText
                  primary={label}
                  sx={{ opacity: open ? 1 : 0 }}
                />
              </ListItemButton>
            </ListItem>
          ))}
        </List>
      </Drawer>
      <Box component="main" sx={{ flexGrow: 1, p: 3 }}>
        <DrawerHeader />
        <Outlet />
      </Box>
    </Box>
  );
};

export default DashLayout;
```



pages/DashboardPages
DashboardPage.jsx

```
import React, { useState } from 'react'
import { useLocation } from 'react-router-dom';
import { BarChart } from '@mui/x-charts/BarChart';

import { DataGrid } from '@mui/x-data-grid';
import Stack from '@mui/material/Stack';
import Box from '@mui/material/Box';

import { Gauge } from '@mui/x-charts/Gauge';
import { Typography, Card, CardContent, Button } from '@mui/material';
import { PieChart } from '@mui/x-charts/PieChart';
import { MapContainer, TileLayer, Marker, Popup } from 'react-leaflet';
import 'leaflet/dist/leaflet.css';

const columns = [
  { field: 'id', headerName: 'ID', width: 90 },
  {
    field: 'firstName',
    headerName: 'First name',
    width: 150,
    editable: true,
  },
  {
    field: 'lastName',
    headerName: 'Last name',
    width: 150,
    editable: true,
  },
  {
    field: 'age',
    headerName: 'Age',
    type: 'number',
    width: 110,
    editable: true,
  },
  {
    field: 'fullName',
    headerName: 'Full name',
    description: 'This column has a value getter and is not sortable.',
    sortable: false,
    width: 160,
    valueGetter: (value, row) => `${row.firstName} ${row.lastName}`,
  },
];

const rows = [
  { id: 1, lastName: 'Snow', firstName: 'Jon', age: 14 },
  { id: 2, lastName: 'Lannister', firstName: 'Cersei', age: 31 },
  { id: 3, lastName: 'Lannister', firstName: 'Jaime', age: 31 },
  { id: 4, lastName: 'Stark', firstName: 'Arya', age: 11 },
  { id: 5, lastName: 'Targaryen', firstName: 'Daenerys', age: null },
  { id: 6, lastName: 'Melisandre', firstName: null, age: 150 },
  { id: 7, lastName: 'Clifford', firstName: 'Ferrara', age: 44 },
  { id: 8, lastName: 'Frances', firstName: 'Rossini', age: 36 },
  { id: 9, lastName: 'Roxie', firstName: 'Harvey', age: 65 },
];

function DashboardPage() {
  const location = useLocation();

  return (
    <Typography variant="h4" gutterBottom>
      Dashboard
    </Typography>
    <Stack direction="column" md="row" spacing={2} sx={{ mb: 4 }} display="flex">
      <Card>
        <CardContent>
          <Typography variant="h6">Total Users</Typography>
          <Typography variant="h4">{rows.length}</Typography>
        </CardContent>
      </Card>
      <Card>
        <CardContent>
          <Typography variant="h6">Average Age</Typography>
          <Typography variant="h4">
            {(
              rows.reduce((sum, row) => sum + (row.age || 0), 0) /
              rows.filter((row) => row.age !== null).length
            ).toFixed(1)}
          </Typography>
        </CardContent>
      </Card>
    </Stack>

    <Stack direction="column" md="row" spacing={3} sx={{ mb: 4 }}>
      <Gauge width={100} height={100} value={50} />
      <Gauge width={100} height={100} value={50} valueMin={10} valueMax={60} />
    </Stack>

    <Stack direction="column" md="row" spacing={3} sx={{ mb: 4 }}>
      <BarChart
        series={[
          { data: [35, 44, 24, 34], label: 'Series 1' },
          { data: [51, 6, 49, 38], label: 'Series 2' },
        ]}
        height={200}
        xAxis={[{ data: ['Q1', 'Q2', 'Q3', 'Q4'], scaleType: 'band', label: 'Quarters' }]}
        title="Quarterly Sales"
      />
      <PieChart
        series={[
          {
            data: [
              { id: 0, value: 10, label: 'series A' },
              { id: 1, value: 15, label: 'series B' },
              { id: 2, value: 20, label: 'series C' },
            ],
          },
        ]}
        width={200}
        height={200}
      />
    </Stack>

    <Stack direction="column" md="row" spacing={3} sx={{ mb: 4 }}>
      <DataGrid
        rows={rows}
        columns={columns}
        experimentalFeatures={{ newEditingApi: true }}
        initialState={{
          pagination: {
            paginationModel: {
              pageSize: 5,
            },
          },
        }}
        pageSizeOptions={[5]}
        checkboxSelection
        disableRowSelectionOnClick
      />
      <React Leaflet Map>
        <Typography variant="h5" gutterBottom sx={{ mt: 4 }}>
          Location Map
        </Typography>
        <Box sx={{ height: 500, width: '100%' }}>
          <MapContainer center={[14.604253, 120.994314]} zoom={13} style={{ height: '100%', width: '100%' }}>
            <TileLayer
              url="https://s.tile.openstreetmap.org/{z}/{x}/{y}.png"
              attribution="©copy; <a href='https://www.openstreetmap.org/copyright'>OpenStreetMap</a> contributors"
            />
            <Marker position={[14.604253, 120.994314]}>
              <Popup>
                National University-Manila <br />
                <p>551 F Jhocson St, Sampaloc, Manila, 1000 Metro Manila</p>
              </Popup>
            </Marker>
          </MapContainer>
        </Box>
      </React Leaflet Map>
    </Stack>
  );
}
```

export default DashboardPage;

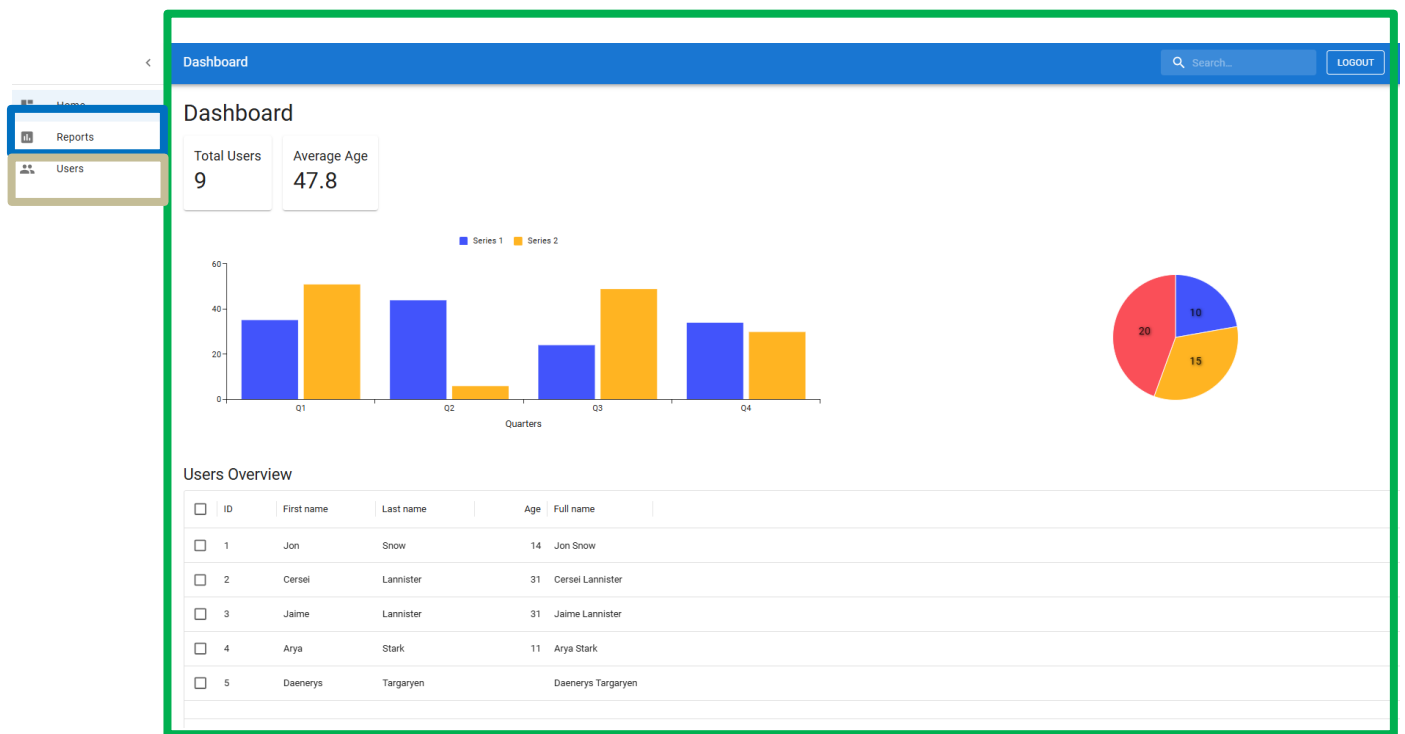
General MUI Implementation Instructions:

1. In installation go to [Installation - Material UI](#)
2. Integrate the **Drawer Component** at the DashLayout or create a component for the navigation to the children of the DashboardPages. Each item list must be linked to the routes e.g.:

```
<ListItemButton component={Link} to="/dashboard" selected={location.pathname === '/dashboard'}>
```

3. In using Charts Components at MUI visit [React Charts – MUI X](#)
4. In using Data Grid [Table] Components at MUI visit [React Data Grid component – MUI X](#)

Sample Output | Enhancement Instructions:



In using MUI:

Enhancement 1: Create and design a **DashboardPage** for [Overview or Summary].

Enhancement 2: Create and design a **ReportsPage** for [Charts or Data Visualization].

Enhancement 3: Create for the **UsersPage** for [User List or Table for Users details].

NOTE: For now, just copy and paste the data included at the MUI Samples for output in preparation for the next activity.

Git Instructions

after the enhancement (must be on root folder surname-webprog)

1. git checkout -b lab-act5
2. git add .
3. git commit -m "lab-act5"
4. git push origin lab-act5

Lab Activity Rubric

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Project Setup & Environment	Project is set up correctly with all dependencies installed. The environment is well-configured and ready for development.	Project is set up with minor issues that do not impact core functionality.	Project setup is attempted but has errors affecting development.	Project setup is incomplete or non-functional.	
Navigation & Routing	Navigation is well-structured, intuitive, and includes a responsive design. Routing is properly implemented with dynamic paths if applicable.	Navigation works correctly but may lack responsiveness or design refinements.	Navigation is functional but may have broken links or a confusing structure.	Navigation is missing or not working.	
Component Structure & Reusability	Components are well-structured, reusable, and follow best practices (e.g., props, state, hooks). Code is modular and scalable.	Components are properly used but may lack optimization or modularity.	Components are used but with poor structure, making the code difficult to scale.	Components are poorly implemented, leading to excessive redundancy or poor organization.	
Styling & UI Design	UI is visually appealing, well-structured, and fully responsive. Consistent styling is applied across the project.	UI is functional and mostly styled well but may have minor inconsistencies or responsiveness issues.	UI is present but lacks proper styling, consistency, or responsiveness.	UI is missing or poorly designed, making navigation difficult.	
Content & Functionality	The app displays dynamic content effectively, and all functionalities work as expected without bugs.	Most functionalities work well, but minor issues exist. Content is displayed correctly.	Some functionalities are incomplete, and content may not load properly.	Major functionalities are missing or broken.	
Code Quality & Best Practices	Code follows best practices, is clean, well-commented, and easy to maintain. Proper use of hooks, state, and props is evident.	Code is well-written but may have minor inefficiencies or inconsistent formatting.	Code runs but is cluttered, inefficient, or difficult to read.	Code is disorganized, hard to understand, and does not follow best practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/24) * 100	